Redis.

01 Qu’est-ce que Redis ?

02 Pourquoi l’utilise-t-on ?

03 Des exemples — et un vrai test face à PostgreSQL sur nos données

```
$ docker exec redis-stack redis-cli ping
PONG
```

Redis Stack 7.4 · PostgreSQL 16 · Python
Mesures du 12 octobre 2026

Une base de données clé-valeur qui vit en mémoire vive

Redis (*REmote DIctionary Server*) range chaque donnée sous une clé unique et garde tout en RAM. Créé en 2009 par Salvatore Sanfilippo, c'est l'une des bases NoSQL les plus répandues.

EN MÉMOIRE	Les lectures ne touchent pas le disque : une commande s'exécute en quelques microsecondes côté serveur.
TYPÉE	Une valeur peut être une chaîne, un hash, une liste, un ensemble trié, un stream… ou un document JSON avec Redis Stack.
PERSISTANTE SI ON LE VEUT	Snapshots périodiques (RDB) et/ou journal des écritures (AOF) sur disque.
DISTRIBUABLE	Réplication, bascule automatique (Sentinel), partitionnement (Cluster).

CLÉ	TYPE	VALEUR
`cache:produits:42`	string	`"{…réponse SQL…}"`
`session:8f3a`	hash	`user_id=42 · panier=3`
`jobs:emails`	list	`[job3, job2, job1]`
`classement`	sorted set	`alice 320 · bob 280`
`obscop2025:row:1`	JSON	`{"Réponse": "Le chômage", …}`

Un seul serveur, plusieurs structures. La dernière ligne est le format de notre projet : une clé par ligne du CSV, la valeur est un document JSON.

Parce qu'il est rapide, simple, et fait plus qu'un cache

01

Vitesse

Tout est en RAM : 3,8 µs de travail par lecture côté serveur. L'écart le plus net est en écriture : une mise à jour ×5 plus rapide que PostgreSQL (1 client).

```
JSON.GET obscop2025:row:1
```

02

Calcul côté serveur

Redis sait incrémenter, trier, empiler. Ces commandes sont atomiques : pas de « lire, modifier, réécrire » dans l'application.

```
ZINCRBY classement 10 alice
```

03

Expiration intégrée

Chaque clé peut avoir une durée de vie (TTL). Idéal pour un cache, une session ou un code à usage unique.

```
SET otp:42 "839201" EX 300
```

04

Simple à adopter

Protocole texte, clients dans tous les langages (redis-py ici), démarrage en une commande Docker.

```
docker run -p 6379:6379 …
```

LES LIMITES

Les données doivent **tenir en RAM** · **un seul cœur** exécute les commandes · pas de SQL ni de jointures · la durabilité dépend de la persistance configurée. Redis vient donc le plus souvent **en complément** d'une base principale, pas à sa place.

Six usages classiques, deux commandes chacun

Cache

STRING + TTL

Garder 60 s le résultat d'une requête SQL ou d'une API lente.

```
SET cache:produits:42 "{…}" EX 60
GET cache:produits:42
```

Sessions

HASH

Session web partagée par plusieurs serveurs, supprimée après 30 min.

```
HSET session:8f3a user_id 42 panier 3
EXPIRE session:8f3a 1800
```

Limitation de débit

INCR

Au plus 100 requêtes par minute et par adresse IP.

```
INCR rl:203.0.113.7    # → 1, 2, 3…
EXPIRE rl:203.0.113.7 60
```

Classement

SORTED SET

Scores d'un jeu, articles les plus lus, mis à jour en temps réel.

```
ZINCRBY classement 10 alice
ZREVRANGE classement 0 9 WITHSCORES
```

File de tâches

LIST

Envoi d'e-mails en arrière-plan. Sidekiq, BullMQ ou Celery peuvent s'appuyer sur Redis.

```
LPUSH jobs:emails "{to: …}"
BRPOP jobs:emails 0   # bloque, attend
```

Temps réel

PUB/SUB

Chat, notifications : les abonnés reçoivent chaque message instantanément.

```
SUBSCRIBE chat:general
PUBLISH chat:general "Bonjour"
```

ObsCop 2025 : une enquête climat dans RedisJSON

Enquête internationale d'opinion sur le changement climatique : 30 pays, 4 thématiques (préoccupations, perception, mobilisation des acteurs, gestes du quotidien).

146 275	21	98 Mo
lignes	colonnes	de CSV

1. Lecture du CSV (UTF-8, séparateur `;`) avec `csv.DictReader`
2. Chaque ligne → un document JSON sous la clé `obscop2025:row:<n>`
3. Écriture par **pipeline** : 500 `JSON.SET` envoyés en un seul aller-retour
4. Lecture directe par clé : `get 1`, `list --start 1 --limit 10`
5. Mêmes lignes chargées dans PostgreSQL pour comparer : `python main.py compare`

MAIN.PY · IMPORT (EXTRAIT SIMPLIFIÉ) — PYTHON MAIN.PY IMPORT

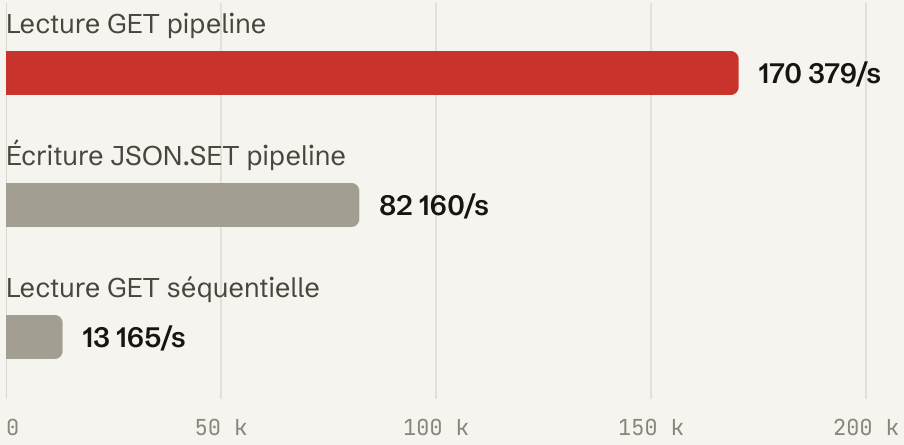
```python
pipeline = client.pipeline(transaction=False)
for n, record in enumerate(records, start=1):
    pipeline.execute_command(
        "JSON.SET", f"obscop2025:row:{n}", "$",
        json.dumps(record, ensure_ascii=False),
    )
    if n % BATCH_SIZE == 0:        # BATCH_SIZE = 500
        pipeline.execute()         # 1 aller-retour / 500 lignes
```

```
$ python main.py get 1
{ "Périmètre spatial": "Monde",
  "Question": "A1BIS. Parmi cette liste, quels sont les sujets…",
  "Réponse": "Le chômage",  "Valeur": "38",  "Unité": "%",
  "Echantillon": "23503",  … 21 champs }
```

Notre premier benchmark mesurait surtout le réseau

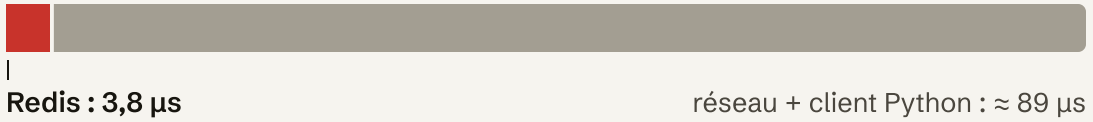
Débit sur 100 000 lignes

opérations par seconde · lots de 500 · médiane de 3 passes



Où passent les 93 µs d'une lecture ?

latence médiane côté client · travail mesuré par `INFO commandstats`



Redis ne travaille que **3,8 µs** par lecture. Le reste, c'est l'aller-retour réseau et le client Python. Le pipeline (×12,9) supprime cette attente : il ne rend pas Redis plus rapide.

La preuve : PostgreSQL gagne autant en lisant par lots de 500 (`WHERE id = ANY(…)`) : **156 756/s**, contre **147 363/s** pour Redis en pipeline.

IMPORT COMPLET

2,8 s

pour les 146 275 lignes, lecture du CSV comprise

MÉMOIRE REDIS

141 Mo

pour 98 Mo de CSV (146 275 clés JSON)

EFFET DU PIPELINE

×12,9

en lecture sur 100 000 lignes : un gain réseau, pas un gain Redis

Face à PostgreSQL indexé, lire une clé : match nul

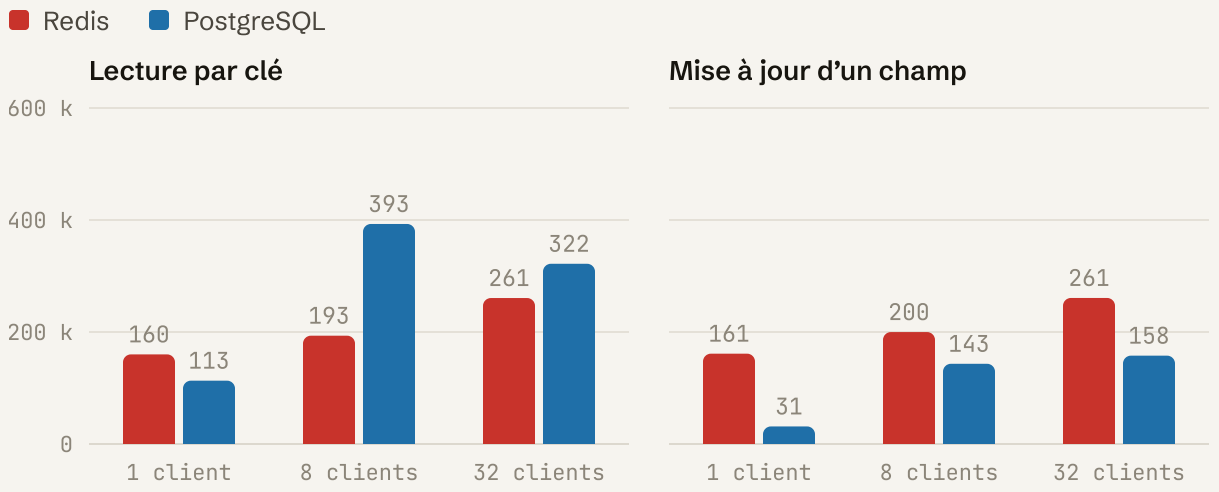
Même client Python, mêmes données

10 000 clés aléatoires · données en mémoire · médiane de 5 passes

	REDIS	POSTGRESQL
Lecture, latence p50	93 µs	82 µs
Lecture, latence p99	114 µs	116 µs
Lecture par lots de 500	147 363/s	156 756/s
32 clients en parallèle	36 709/s	32 245/s
Travail serveur par lecture	3,8 µs	5,4 µs
Import des 146 275 lignes	1,83 s pipeline	1,03 s COPY

Capacité des serveurs seuls

milliers d'opérations/s selon le nombre de clients · redis-benchmark et pgbench dans les conteneurs

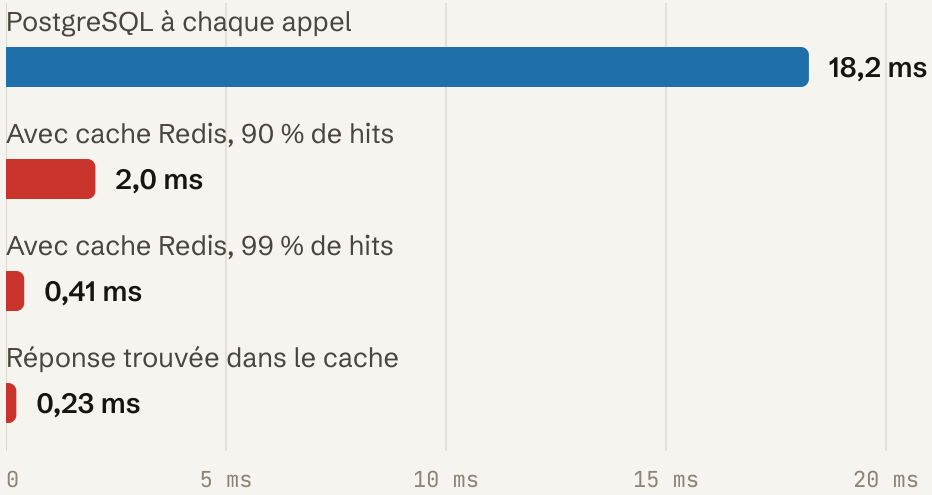


Redis exécute toutes les commandes sur **un seul cœur** : il plafonne vers 261 k/s. PostgreSQL lance un processus par connexion et profite des 10 cœurs en lecture. En écriture, Redis reste devant.

Le vrai gain : ne pas recalculer

Tableau de bord : préoccupations par pays (question A1BIS)

740 lignes extraites des 146 275 · temps par appel en ms · médiane



CACHE « À LA DEMANDE » (CACHE-ASIDE), SIMPLIFIÉ

```
def tableau_de_bord():
    valeur = redis.get(CLE)                  # 0,23 ms
    if valeur is not None:
        return json.loads(valeur)
    lignes = pg.execute(REQUETE).fetchall()  # 18,2 ms
    redis.set(CLE, json.dumps(lignes), ex=60)  # TTL 60 s
    return lignes
```

×78 quand la réponse est déjà en cache. En moyenne, ×9 si 90 % des appels la trouvent, ×44 à 99 %.

PostgreSQL passe 18,3 ms à parcourir les 146 275 documents. Redis ne fait que rendre le résultat déjà calculé.

En résumé

01 · Qu’est-ce que c’est ?

Une base clé-valeur en mémoire

Des valeurs typées (chaînes, hashes, listes, ensembles triés, streams, JSON), des commandes atomiques, une persistance optionnelle sur disque.

COMMANDES VUES

`SET · GET · HSET · ZADD · JSON.SET`

02 · Pourquoi ?

Pas pour lire plus vite qu’une base indexée

Sur des données en mémoire, PostgreSQL fait jeu égal. Redis sert à **éviter de recalculer**, à absorber des **écritures légères** (sessions, compteurs) et à profiter du TTL et de ses structures.

MESURÉ

`cache ×78 · mise à jour ×5 (1 client)`

03 · Exemples

Cache, sessions, files, temps réel

Chez nous : le tableau de bord A1BIS servi en **0,23 ms** au lieu de **18,2 ms**, et 146 275 lignes tenues dans **141 Mo de RAM**.

À RETENIR

`Redis complète PostgreSQL, il ne le remplace pas`

CONDITIONS DE TEST

MacBook Apple M4 (10 cœurs) · Redis Stack 7.4.7 et PostgreSQL 16.15 dans Docker (OrbStack), en local · Python 3.9.6, redis-py 5.0.8, psycopg 3.2.13 · PostgreSQL : `synchronous_commit=off`, `shared_buffers=256MB` · données en mémoire · médiane de 5 passes alternées. Machine de travail : jusqu’à ±30 % d’écart entre passes, à lire comme des ordres de grandeur.